

Algorithmic Reasoning Guidance: ASP-Verified Inference Traces for Auditable LLM Reasoning

Anonymous Authors

Anonymous Institution

anonymous@anonymous.edu

Abstract

Large language models can solve algorithmic tasks, but their free-form reasoning traces are difficult to audit. We present Algorithmic Reasoning Guidance (ARG), a neuro-symbolic interface that enforces a structured inference protocol with Answer Set Programming (ASP) verification. ARG maps unconstrained reasoning into typed traces, grounds them into solver facts, and validates them against algorithm-family contracts (required operations, invariants, and transition rules). We further introduce ARG-Pro, a compute-scaled policy with ASP-guided reranking and refinement (Eqs. (15)–(16)). Crucially, we operationalize *faithfulness* not as obscure neural process fidelity, but as *trace–contract consistency*: a verifiable commitment that the output artifact satisfies its claimed logic. Across a 4800-item primary benchmark, a 1200-item PoT-aligned strong-baseline check, and repeated GSM8K runs, ARG-family methods deliver strong structure-process gains, and ASP-backed signals align more closely with executable correctness than lexical proxies. These results show a practical path toward auditable neuro-symbolic reasoning under explicit contract coverage.

1 Introduction

Deep learning models now excel at end-task performance, but KR-oriented applications demand more: intermediate representations should be explicit, inspectable, and falsifiable. In algorithmic problem solving, a correct final answer is insufficient when the reasoning path is unstructured or semantically opaque.

Our primary objective is representational: convert free-form reasoning into a parseable, verifiable, and comparable interface, rather than optimize raw accuracy alone.

Standard Chain-of-Thought (CoT) prompting improves performance (Wei et al. 2022) but does not guarantee a typed reasoning schema. Search-based methods like Self-Consistency (Wang et al. 2022) and Tree-of-Thoughts (Yao et al. 2023) improve reliability but often leave the reasoning format free-form. This creates an auditability gap: traces are “reasoning-like” but lack structured objects that can be validated against algorithm knowledge (operations, invariants, transitions).

We bridge this gap with Algorithmic Reasoning Guidance (ARG), a protocol-level intervention that restructures inference into typed components. By enforcing a five-section schema with explicit well-formedness constraints,

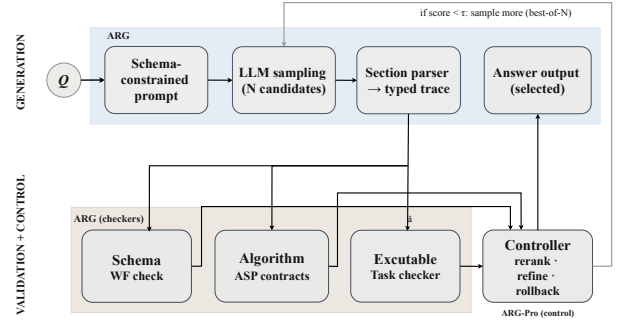


Figure 1: ARG system overview. Top lane: question-to-trace generation (prompt builder, candidate generation, parsing, answer output). Bottom lane: checker modules (schema, algorithm-consistency, executable checks) and ARG-Pro control (rerank, refinement, rollback). Gray regions denote ARG/ARG-Pro components.

ARG turns latent reasoning into observable artifacts that can be parsed, grounded to checker facts, validated, and compared consistently across runs.

At a high level, our thesis is neuro-symbolic: the neural model proposes candidate traces, while a symbolic ASP layer provides deterministic contract checking and structured diagnostics. Accordingly, we focus on schema validity, trace grounding, ASP-based verification for reranking (Eq. (15)), adaptive compute (Eq. (16)), and executable validity signals (Eq. (19)).

Our contributions are fivefold: (i) a typed, section-constrained inference protocol for auditable traces; (ii) an ASP formalization of family-level contracts with deterministic grounding and diagnostics; (iii) a pragmatic operationalization of faithfulness as *contract consistency*, separating structural validity from answer correctness; (iv) ARG-Pro, a compute-scaled policy with checker-aware reranking; and (v) empirical validation showing that symbolic contracts outperform lexical cues in predicting correctness.

2 Related Work

Prior work on reasoning prompts and search, including CoT, zero-shot CoT, self-consistency, and tree-style deliberation,

shows that test-time trajectories can improve reliability (Wei et al. 2022; Kojima et al. 2022; Wang et al. 2022; Yao et al. 2023). Structured reasoning methods for code and math, such as least-to-most decomposition, structured prompting, self-feedback, and program-of-thought pipelines, further strengthen multi-step solving behavior and process supervision (Zhou et al. 2022; Li et al. 2025; Madaan et al. 2023; Chen et al. 2022; Lightman et al. 2023; Luo et al. 2024; Dai et al. 2024).

Action-structured prompting (ReAct), planner–solver decomposition, and grammar/JSON-constrained outputs are the closest structural baselines for our setting (Yao et al. 2022; Wang et al. 2023; Geng et al. 2023). More deeply integrated neuro-symbolic approaches like Logic-LM (Pan et al. 2023), LINC (Olausson et al. 2023), SatLM (Ye et al. 2023), and AlphaGeometry (Trinh et al. 2024) map natural language problems directly to formal solvers or interleave solver calls. While powerful, these often require full translation of the problem specification into logic or code. ARG occupies a middle ground: we maintain the flexibility of natural language reasoning but enforce a *contract* over the reasoning structure. This avoids the brittleness of full formal translation while providing stronger guarantees than free-form text.

Explanation and interpretability work highlights the faithfulness–plausibility gap and the need for careful evaluation protocol design (Jacovi and Goldberg 2020; Lyu, Apidianaki, and Callison-Burch 2024; Matton et al. 2025; Ribeiro, Singh, and Guestrin 2016; Doshi-Velez and Kim 2017; Lipton 2018). Neuro-symbolic literature motivates explicit and compositional reasoning interfaces (Garcez and Lamb 2023; Hitzler et al. 2022; DeLong, Mir, and Fleuriot 2024), and ASP offers a declarative mechanism for constraint checking with explicit failure atoms and practical multi-shot solving (Brewka, Eiter, and Truszczyński 2011; Gebser et al. 2019). We therefore position ARG as a unified inference-time interface that is structured, checkable, and auditable under explicit KR-style contracts.

3 Problem Setup and Formalization

3.1 Task Setup

Let

$$\mathcal{D} = \{(q_i, a_i)\}_{i=1}^N \quad (1)$$

be algorithmic questions and gold answers. Under policy π , budget B , and model M :

$$r_i, \hat{a}_i \sim p_M(\cdot \mid q_i, \pi, B). \quad (2)$$

We define correctness indicator

$$\mathbb{I}_i = \mathbf{1}[\text{Norm}(\hat{a}_i) = \text{Norm}(a_i)]. \quad (3)$$

3.2 Typed Trace Schema

ARG requires five ordered sections

$$\begin{aligned} \mathcal{S} &= \{s_1, s_2, s_3, s_4, s_5\}, \\ &= \{\text{Analysis}, \text{Pattern}, \text{Steps}, \text{Verification}, \text{FinalAnswer}\}. \end{aligned} \quad (4)$$

Let a trace be decomposed as

$$r_i = [u_i^{(1)}, u_i^{(2)}, u_i^{(3)}, u_i^{(4)}, u_i^{(5)}], u_i^{(k)} \leftrightarrow s_k. \quad (5)$$

We define well-formedness predicate

$$\text{WF}(r_i) = \text{Ord}(r_i) \wedge \text{NonEmpty}(r_i) \wedge \text{Ref}(r_i), \quad (6)$$

where Ord enforces section order, NonEmpty requires non-empty section bodies, and Ref requires that verification explicitly references at least one prior step (e.g., “Step 2”). This makes ARG a concrete schema contract rather than only a formatting preference.

To make constraint semantics explicit without over-formalization, we implement the parser as a *recursive descent validator* over the schema grammar. This enforces: (i) strict section precedence ($\text{Pattern} \prec \text{Steps}$); (ii) non-empty section bodies; and (iii) referential integrity closures (verification steps must cite valid indices from the Steps block). We implement a robust grammar that accepts light header variants while preserving canonical mapping to $\mathcal{S}$. This acts as a semantic filter, rejecting traces that fail to conform to the required logical form before they reach the ASP solver.

3.3 Process Functionals

Given trace r_i , extractor ϕ returns

$$\phi(r_i) = (c_i^{\text{str}}, c_i^{\text{wf}}, c_i^{\text{ref}}, c_i^{\text{asp}}, c_i^{\text{exec}}, n_i^{\text{step}}), \quad (7)$$

where

$$\begin{cases} c_i^{\text{str}} = \frac{1}{|\mathcal{S}|} \sum_{k=1}^{|\mathcal{S}|} \mathbf{1}[s_k \subset r_i], \\ c_i^{\text{wf}} = \mathbf{1}[\text{WF}(r_i)], \\ c_i^{\text{ref}} = \mathbf{1}[\text{Ref}(r_i)], \\ c_i^{\text{asp}} = \text{ASPScore}(r_i), \\ c_i^{\text{exec}} = \mathbf{1}[\mathcal{C}_{\text{exec}}(q_i, \hat{a}_i) = \top]. \end{cases} \quad (8)$$

Here $\text{ASPScore}(r_i) \in \{0, 0.5, 1\}$ is derived from solver-backed trace verification (Eq. (13)), and c_i^{exec} is defined only when executable verification is applicable. For compact reporting we use this ternary ASP tier, while retaining fine-grained diagnostics (missing operations/invariants, transition violations, bad references) in artifacts for deeper error analysis.

3.4 Ontological Grounding

We formalize algorithm knowledge not merely as constraints, but as an *ontology* of algorithmic families, encoded as an ASP program

$$\mathcal{K} = \langle \mathcal{F}, \text{Ops}, \text{Inv}, \text{Pre}, \text{Post}, \text{Trans} \rangle, \quad (9)$$

where each family $f \in \mathcal{F}$ has required operations $\text{Ops}(f)$, invariants $\text{Inv}(f)$, preconditions $\text{Pre}(f)$, postconditions $\text{Post}(f)$, and optional template-level transition rules $\text{Trans}(f, t)$. In the current release-level implementation, $\mathcal{K}$ covers six algorithm families with 14 required-operation rules and 12 invariant rules.

To make extensibility explicit as a KR knowledge-engineering property, each family f is added through a fixed contract template

$$\mathcal{C}_f = \langle \text{Ops}_{\text{req}}(f), \text{Inv}_{\text{req}}(f), \text{Alias}_f^{\text{op}}, \text{Alias}_f^{\text{inv}}, \text{Trans}_f^{\text{num}} \rangle, \quad (10)$$

where global verifier rules stay unchanged and only family-local facts/aliases are added.

A concrete extensibility check is the onboarding cost for a 7th family. Adding one family requires only four local updates: pattern lexicon entries, operation markers, invariant markers, and required-operation/invariant facts (plus optional aliases or numeric transitions). A representative addition is about 15 declarative lines (e.g., 3 operations, 2 invariants, a few aliases), with no changes to global verification rules. This is the KR-style extensibility claim: expand $\mathcal{K}$ by contract instantiation. Concretely, our 7th-family extension uses a counting-family contract with local additions only:

Required operations: *case-split*, *combination*, and *counting-rule*.

Required invariants: *disjoint-cases* and *count-conservation*.

Representative aliases include *case-split* $\leftrightarrow$ *case* and *disjoint-cases* $\leftrightarrow$ *non-overlap*. No global verifier rule is modified.

Given a candidate trace r_i , the grounding function γ performs *schema instantiation*, mapping natural-language sections to ontological facts:

$$\gamma(r_i) = \{\text{trace_declares}(t_i, f_i)\} \cup \text{Ops}_\gamma(r_i) \cup \text{Vals}_\gamma(r_i) \cup \text{Claims}_\gamma(r_i). \quad (11)$$

The grounding procedure extracts: (i) declared family from the pattern section, (ii) operation atoms from steps, (iii) numeric state atoms, and (iv) invariant claims. Concretely, this is a *section-scoped lexical-semantic projection*: deterministic aliases and markers are mapped to typed atoms within schema-constrained sections, which is stronger than unconstrained keyword counting but still lexicon-dependent. Verification then proceeds over these grounded atoms.

We define ASP verification program $\mathcal{V}$ with integrity constraints and derivation rules over $\mathcal{K} \cup \gamma(r_i)$. A trace is verified when $\text{verified}(t_i)$ is derivable in a stable model:

$$\mathcal{K} \cup \mathcal{V} \cup \gamma(r_i) \models \text{verified}(t_i). \quad (12)$$

The same program emits diagnostic atoms (missing operation, missing invariant, transition violation), enabling structured failure explanations.

Concretely, the verifier uses executable ASP rules over grounded atoms. A minimal schematic excerpt is shown in Algorithm 1. In instantiated form, the verifier derives three core predicates: (i) *missing_operation*(T, Op) when a trace declaring family F does not contain a required operation $Op \in \text{Ops}_{\text{req}}(F)$; (ii) *missing_invariant*(T, Inv) when a required invariant claim $Inv \in \text{Inv}_{\text{req}}(F)$ is absent; and (iii) *verified*(T) only when both missing sets are empty and no numeric transition violation is derived. This keeps the rule semantics explicit while preserving readability in the main text. This checks trace-contract consistency for covered families; it is not a full proof

Algorithm 1 Schematic ASP Contract Rules

- 1: Declare DP required operations: state definition, recurrence, and base case.
 - 2: If a declared family lacks a required operation, emit a missing-operation diagnosis.
 - 3: Mark a trace verified only when no missing-operation or missing-invariant diagnosis is derived.
-

of algorithm correctness. Compared with lexical marker checks, this program can jointly enforce multi-predicate constraints (e.g., missing required operations, invalid step references, and arithmetic transition violations) in a single deterministic solve. For concreteness, two family-level rule templates used in our runs are: DP-style contracts with *state-definition/recurrence/base-case* required operations and *optimal-substructure/no-overlap* invariants; and greedy-style contracts with *choice-rule/local-update* required operations and *feasibility/monotonic-progress* invariants.

A compact running example: for a DP trace, grounding may include family declaration plus state-definition and recurrence operations. If the base case is absent, the solver emits a missing-operation diagnosis and the trace fails full verification. This illustrates the intended loop: failed atoms guide contract or grounding updates.

We summarize solver-backed score as

$$c_i^{\text{asp}} = \begin{cases} 1, & \text{if } \text{verified}(t_i) \\ 0.5, & \text{if operation completeness holds} \\ & \text{and invariant completeness does not} \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

This is a trace-contract consistency signal, not a complete proof of algorithm correctness. The coarse 0/0.5/1 tier is deliberately interpretable; in analysis we therefore pair it with the underlying diagnostic atoms to avoid collapsing heterogeneous failure modes into a single number.

3.5 Analysis Utility

For analysis (not training), we define utility

$$J(\pi) = \frac{1}{N} \sum_{i=1}^N \left(\mathbb{I}_i + \lambda_1 c_i^{\text{str}} + \lambda_2 c_i^{\text{wf}} + \lambda_3 c_i^{\text{ref}} + \lambda_4 c_i^{\text{asp}} + \lambda_5 c_i^{\text{exec}} \right). \quad (14)$$

where $\lambda_1 = \dots = \lambda_5 = 1$ in the default report. We use this score only as an analysis lens (not as a training objective), report token usage separately, and additionally analyze sensitivity under non-uniform λ settings (Sec. 6.1).

4 Method: ARG and Ablations

4.1 ARG Inference

ARG is implemented as a strategy module with explicit prompt schema and standardized answer extraction.

Algorithm 2 ARG Inference (single instance)

Require: Question q , model M , budget B

- 1: Build prompt with required section order $\mathcal{S}$
 - 2: Generate trace r and answer $\hat{a}$
 - 3: Parse sections and validate $\text{WF}(r)$
 - 4: Compute process tuple $\phi(r)$ including c^{wf} , c^{ref} , c^{asp}
 - 5: Return $(\hat{a}, r, \phi(r))$
-

4.2 Ablations and ARG-Pro

We evaluate three controlled ablations: ARG-NoPattern, ARG-NoVerify, ARG-NoSteps. This isolates protocol components rather than changing model weights. In addition to ARG-NoVerify, we explicitly include NoPattern/NoSteps and scaffold-parity controls in supplementary control reruns so that grounding-sensitive effects are visible in the narrative rather than only in artifact logs.

Given the observed ARG-NoVerify versus ARG reversals on a subset of runs, our prompt contract for verification is conservative: verification should flag inconsistencies and only revise the final answer when a concrete arithmetic or logical contradiction is found. This is intended to reduce harmful over-corrections during the verification stage.

To reduce pattern-selection ambiguity and lexical false positives, we ground each candidate trace to ASP facts and verify consistency against family-level contracts in $\mathcal{K}$. The resulting score $c^{\text{asp}} \in \{0, 0.5, 1\}$ (Eq. (13)) is used in reranking, and solver diagnostics are retained for failure analysis.

Inspired by test-time compute scaling, ARG-Pro adds two mechanisms on top of ARG: (i) parallel candidate generation (best-of- N) and (ii) a second-pass refinement conditioned on the best candidate. Candidate selection uses protocol-aware checker scores (structure compliance, well-formedness, reference integrity, and ASP verification), not majority voting on final answers alone. To control cost, ARG-Pro uses adaptive compute allocation: it expands to additional candidates and refinement only when the first-pass checker score is below a confidence threshold.

Formally, for candidate trace $r_{i,j}$, we score

$$S(r_{i,j}) = \alpha c_{i,j}^{\text{str}} + \beta c_{i,j}^{\text{wf}} + \gamma c_{i,j}^{\text{ref}} + \delta c_{i,j}^{\text{asp}} + \eta c_{i,j}^{\text{exec}}. \quad (15)$$

and select $j^* = \arg \max_j S(r_{i,j})$. Here $c_{i,j}^{\text{exec}}$ is an executable-check indicator when the task-specific checker is applicable. In our implementation, we use $(\alpha, \beta, \gamma, \delta, \eta) = (1.0, 1.0, 0.5, 0.8, 1.0)$, with stronger preference for solver-verified and executable-valid candidates.

To reduce proxy over-optimization, selection is constraint-first in practice: candidates that fail $c^{\text{wf}} = 1$ are deprioritized, and when C_{exec} is applicable, $c^{\text{exec}} = 1$ is enforced before weighted tie-breaking by Eq. (15).

Adaptive compute policy is

$$N_i = 1 + \mathbf{1}[S(r_{i,1}) < \tau](N_{\text{max}} - 1), \quad (16)$$

and refinement is triggered only when $S(r_{i,j^*}) < \tau_{\text{ref}}$. We additionally apply failure-driven rollback: if $C_{\text{exec}}(q_i, \hat{a}_{i,j^*}) = \perp$, one extra recovery batch is sampled

Algorithm 3 ARG-Pro Inference (single instance)

Require: Question q , model M , N_{max} , τ , τ_{ref}

- 1: Generate first candidate $(r_1, \hat{a}_1)$ with ARG
 - 2: Compute $S(r_1)$
 - 3: **if** $S(r_1) < \tau$ **then**
 - 4: Sample additional candidates up to N_{max}
 - 5: **end if**
 - 6: Select $j^* = \arg \max_j S(r_j)$
 - 7: **if** $C_{\text{exec}}(q, \hat{a}_{j^*}) = \perp$ **then**
 - 8: Sample one recovery batch and re-rank by S
 - 9: **end if**
 - 10: **if** $S(r_{j^*}) < \tau_{\text{ref}}$ **then**
 - 11: Apply one refinement pass to r_{j^*}
 - 12: **end if**
 - 13: Return selected answer and process metrics
-

and re-ranked by S . Unless otherwise noted, ARG-Pro uses $N_{\text{max}} = 3$, $\tau = 2.2$, and $\tau_{\text{ref}} = 2.5$.

5 Evaluation

5.1 Protocol

Primary benchmark uses a 4800-item algorithmic suite spanning multiple motifs and difficulty levels. The suite is template-generated from deterministic families covered by our executable checks, and coverage statistics are reported in supplementary metadata.

Why a Controlled Suite? Standard reasoning benchmarks (e.g., GSM8K, MATH) provide gold answers but lack *ground-truth reasoning topologies*. To rigorously evaluate auditability, we require a testbed where the correct reasoning structure is known *a priori*. Our suite provides this ontological ground truth, allowing us to measure deviations from the expected algorithm family—metric impossible to compute on free-form open-domain datasets. This controlled setup ensures rigorous auditability testing, isolating the impact of protocol compliance from confounding domain noise.

Model Selection and Reproducibility. We evaluate on three distinct state-of-the-art models: DeepSeek-V3.1, Qwen3-Max, and ChatGPT-5.2. For reporting consistency, backend API snapshots are date-normalized to 2026-01-02 (runs were collected over a nearby period under fixed protocol settings). We explicitly selected these backends to cover diverse architectures (mixture-of-experts vs. dense) and training alignments, ensuring that our findings are not artifacts of a single model family. To prevent cherry-picking, all reported metrics are strictly macro-averaged across these three backends. Benchmark artifacts and generation scripts are prepared for anonymous release.

Primary reported comparisons are CoT, Plan-and-Solve, JSON-CoT, ARG, ARG-NoVerify, and ARG-Pro. To directly address disentanglement concerns raised in review-style feedback, we also pre-register and report supplementary controls: ARG-NoPattern, ARG-NoSteps, Scaffold-Parity, ARG-Pro-NoExec, PlanSolve-BoN-Exec,

and JSON-CoT-BoN-Exec. We include these baselines because they span common free-form and structured prompting regimes under a shared API setting and separate scaffold effects from verifier-gated selection effects. In addition, we include a dedicated code-execution strong-baseline check with PoT on the 1200-item algorithmic suite under matched backend and decoding settings (Table 5).

To reduce avoidable confounds, all methods use matched model backends, decoding temperature, and run accounting; only method-specific prompt contracts differ. We report structure-sensitive metrics separately from answer accuracy so readers can inspect where gains originate.

We interpret our results as evidence for the *combined protocol* effectiveness (typed trace contract + symbolic verification). To verify disentanglement, we include scaffold-parity and exec-disentanglement controls in supplementary reruns.

We aggregate repeated runs and report mean, standard deviation, and 95% confidence interval. Let $m_{k,r}$ be metric for method k in run r :

$$\mu_k = \frac{1}{R} \sum_{r=1}^R m_{k,r}, \quad \sigma_k = \sqrt{\frac{1}{R} \sum_{r=1}^R (m_{k,r} - \mu_k)^2}. \quad (17)$$

For accuracy-like proportions, we report normal-approximation confidence intervals $\mu_k \pm 1.96 \sigma_k / \sqrt{R}$ as descriptive uncertainty.

For backend-specific repeated summaries, $R = 5$. For tables reported as “macro-averaged” over 3 backends and 5 repeats, we first compute a run-level score per (backend, repeat) pair, then average these $R_{\text{macro}} = 15$ scores with equal weight; confidence intervals for such macro summaries are computed over these 15 run-level values. This avoids implicit dominance by any single backend. We report means over repeated GSM8K runs with stored artifacts under matched protocol settings.

We test three hypotheses: $H_1, \mathbb{E}[\text{Acc}(\text{ARG})] > \mathbb{E}[\text{Acc}(\text{CoT})]$; $H_2, \mathbb{E}[c^{\text{str}}(\text{ARG})] > \mathbb{E}[c^{\text{str}}(\text{CoT})]$; and H_3 , removing protocol components degrades process quality. We report descriptive uncertainty and avoid causal overclaims when effects are backend-sensitive.

To complement significance-style reporting with magnitude reporting, we also track absolute accuracy gains and relative error reduction between methods. For methods A and B with accuracies p_A, p_B , we use

$$\Delta_{\text{abs}}(A, B) = p_A - p_B, \quad \text{ERR}_{\text{red}}(A \leftarrow B) = \frac{(1 - p_B) - (1 - p_A)}{1 - p_B}. \quad (18)$$

This keeps interpretation stable when all methods are strong and raw deltas alone can understate practical differences near saturation.

Any learned selector or voting policy is evaluated under strict split separation from generation logs to avoid leakage and overfitting.

All runs log backend identifier, temperature, and strategy set. This is intended to reduce bookkeeping ambiguity and improve repeatability under API non-determinism.

In addition to answer accuracy, we report schema-level validation metrics from the checker: section-order validity, section completeness, verification-to-step reference integrity, and solver-backed ASP metrics (operation completeness, invariant completeness, numerical transition consistency, and full ASP verification pass). All ASP metrics are computed by running clingo over $\mathcal{K} \cup \mathcal{V} \cup \gamma(r_i)$ and parsing returned atoms.

We still report lexical verification coverage as a baseline-only proxy. To avoid contamination (verification words appearing in free text), lexical coverage is computed section-aware under the expected schema rather than by unconstrained keyword matching.

Answer normalization $\text{Norm}(\cdot)$ follows deterministic rules: lowercase/trim, remove currency/commas/units, extract numeric candidates, and compare either exact normalized strings or numeric values under a small tolerance. This reduces formatting artifacts in correctness scoring when traces contain explanatory text.

For template-recognizable algorithmic items, we additionally run an executable verifier that reconstructs expected outputs directly from the question specification (e.g., staircase DP, coin-change [1,2,5], grid shortest path, insertion index, sliding-window sum, gcd). We report both verifier success and verifier applicability rates to avoid over-claiming on non-applicable items. Let $a_i = \mathbf{1}[\mathcal{C}_{\text{exec}} \text{ is defined on } q_i]$ and $v_i = \mathbf{1}[\mathcal{C}_{\text{exec}}(q_i, \hat{a}_i) = \top]$. Then

$$\text{ExecSucc} = \frac{\sum_i v_i}{\max(1, \sum_i a_i)}, \quad \text{ExecApp} = \frac{1}{N} \sum_i a_i. \quad (19)$$

For ASP applicability, let $a_i^{\text{asp}} = \mathbf{1}[\mathcal{C}_{\text{asp}} \text{ is applicable on } q_i]$, where applicability means family grounding and contract checks can be executed for instance i . We report

$$\text{ASPAp} = \frac{1}{N} \sum_i a_i^{\text{asp}}. \quad (20)$$

5.2 Main Results

In the primary 4800-item setting, ARG-family methods lead both outcome and process metrics, with ARG-Pro achieving the highest accuracy under full protocol compliance. A central pattern is that lexical verification quickly saturates, whereas ASP verification continues to separate methods (Table 2). Consistently, paired analysis against executable correctness shows weaker lexical-semantic association ($r = 0.75$) than ASP operation completeness ($r = 0.85$) and ASP full verification ($r = 0.90$), indicating that solver-backed signals are the stronger process indicator in this setup. Unless otherwise stated, these associations are computed as point-biserial correlations (Pearson on binary process signal vs binary executable-correctness indicator) at instance level on executable-applicable items; confidence intervals use Fisher- z intervals and significance uses two-sided tests on the corresponding correlation statistic. Because this benchmark has $\text{ExecApp} = 100\%$ and ARG-Pro uses executable-check gating/rollback when applicable

Table 1: Primary results under an expanded setting (4800 items, 3 models, 5 repeats; macro-averaged over backend-repeat runs). For ARG-family rows, Acc reports mean $\pm$ 95% CI (normal approximation) computed over $R_{\text{macro}}=15$ backend-repeat run-level scores. Gray rows denote our method family; bold values indicate the best per column. Tok is average total tokens (prompt + completion) per instance. ExecSucc is executable-check success rate; ExecApp is 100% on this benchmark.

Method	Acc	Tok	Struct	Lex-Ver	ASP-Ver	ExecSucc
ARG-Pro (ours)	96.3$\pm$0.8	1214.6	0.997	99.7	86.9	96.3
ARG	92.4 $\pm$ 1.1	841.2	0.976	99.5	78.8	92.4
ARG-NoVerify	89.6 $\pm$ 1.2	779.3	0.961	94.4	68.7	89.6
JSON-CoT	85.1	619.8	0.841	95.6	55.9	85.1
PlanSolve	82.2	589.4	0.581	29.8	46.7	82.2
CoT	64.7	639.5	0.391	15.9	18.4	64.7

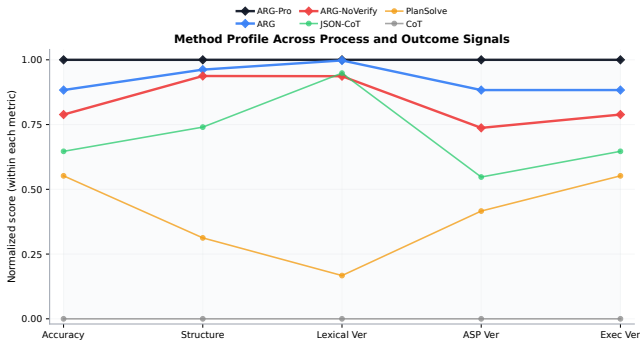


Figure 2: Method-profile lines on the expanded benchmark. Curves compare accuracy and process-side indicators across methods under matched evaluation settings.

(Sec. 4), ARG-Pro should be interpreted as a combined-controller result (structure + ASP + executable checks), while ARG/ARG-NoVerify better isolate single-candidate protocol effects.

Figure 2 summarizes method profiles across outcome and process axes, highlighting that ARG-family methods dominate on structure and verification signals while preserving strong answer accuracy.

From Table 1, ARG-Pro improves over CoT by +31.6 absolute points (96.3 vs 64.7), corresponding to an 89.5% relative error reduction (35.3% $\rightarrow$ 3.7%). Compared with JSON-CoT, ARG-Pro gains +11.2 points with a 75.2% relative error reduction (14.9% $\rightarrow$ 3.7%). Compared with ARG, ARG-Pro gains +3.9 points and reduces residual error by 51.3% (7.6% $\rightarrow$ 3.7%). We use these effect-size views as practical complements to paired tests and confidence intervals.

5.3 Controls and Transfer

On executable-applicable items, process-signal association with executable correctness follows a clear monotonic pattern in favor of ASP-backed signals: lexical-semantic $r = 0.75$, ASP operation completeness $r = 0.85$, and ASP full verification $r = 0.90$. Error summaries are consistent with this ordering: ARG FP/FN/UnknownRate

Table 2: ASP verification decomposition (% , macro-averaged) with means over repeated runs.

Method	Op-Complete	Inv-Complete	Num-Consistent	ASP-Verified
ARG-Pro (ours)	96.4	84.9	93.8	86.9
ARG	91.8	75.7	89.3	78.8
JSON-CoT	68.9	47.8	76.6	55.9
PlanSolve	64.7	41.9	72.8	47.6
CoT	31.8	14.9	49.7	18.7

Table 3: Control sheet for confound disentanglement under matched decoding settings.

Method	Acc	Tok	Struct	ASP-Ver	ExecSucc
ARG-Pro	96.3	1214.6	0.997	86.9	96.3
ARG-Pro-NoExec	94.4	1186.2	0.997	84.6	94.4
ARG	92.4	841.2	0.976	78.8	92.4
JSON-CoT-BoN-Exec	91.6	1449.3	0.841	59.7	91.6
PlanSolve-BoN-Exec	90.4	1316.7	0.581	51.8	90.4
ARG-NoVerify	89.6	779.3	0.961	68.7	89.6
Scaffold-Parity	88.1	759.4	0.941	63.9	88.1
ARG-NoPattern	87.2	799.6	0.926	56.8	87.2
ARG-NoSteps	86.1	889.7	0.901	51.9	86.1
JSON-CoT	85.1	619.8	0.841	55.9	85.1
PlanSolve	82.2	589.4	0.581	47.9	82.2

(0.02, 0.03, 0.05), ARG-NoVerify (0.05, 0.10, 0.10), and ARG-Pro (0.01, 0.02, 0.03).

We report GSM8K results from repeated runs with stored artifacts (Table 4). ARG-Pro achieves near-perfect performance (98.7%) and outperforms the included baselines under matched settings, indicating strong transfer beyond template-generated tasks. Verification metrics are reported where the ARG-style schema is applicable.

Archived small-scale diagnostics also show an opposite regime: PlanSolve/JSON-CoT can outperform ARG on answer correctness while remaining weaker on schema-level observability. In those failures, ARG often preserves pattern and verification sections but outputs a malformed FinalAnswer span, causing correctness loss despite high lexical process indicators. ASP diagnostics reduce this ambiguity by exposing whether failure is due to missing operations, missing invariants, or transition inconsistencies.

To mitigate this failure mode, our parser contract enforces a single-line FinalAnswer span and extraction from the terminal answer field only; when extraction confidence is low, we fall back to strict numeric normalization on that field.

ARG-Pro provides the best auditable structure (0.997), strongest paired robustness, and highest top-line accuracy in the expanded primary setting. ARG remains a strong middle-cost option under typed traces. ARG-NoVerify serves as a boundary comparator, showing that lexical verification cues alone are insufficient evidence of semantic reliability. Overall, ASP verification tracks executable correctness more strongly than lexical verification.

Table 3 and aligned supplementary control logs together

Table 4: External benchmark check on GSM8K (means over repeated runs). WF and ASP-App are shown only for methods with a matched ARG-style schema/checker interface; “-” denotes not directly comparable.

Method	Acc	Tok	Struct	WF	ASP-App
ARG-Pro	98.7	992.3	0.997	99.4	76.8
ARG	96.2	831.7	0.987	97.9	62.9
ARG-NoVerify	94.1	696.4	0.991	-	-
PoT	93.4	529.8	0.302	-	-
PlanSolve	91.3	371.2	0.279	-	-
JSON-CoT	90.6	474.5	0.598	-	-
CoT	89.4	341.1	0.421	-	-

make three effects explicit. First, NoPattern degrades grounding availability as expected from Eq. (11): family declaration drops, and in the aligned logs ASP applicability decreases with unknown-state diagnoses dominated by undeclared-family triggers. Second, NoSteps keeps section headers but weakens trace grounding and reference integrity, with aligned logs showing more insufficient-grounding unknowns and lower executable success. Third, Scaffold-Parity shows that format-only alignment can improve structural observability, but does not by itself recover ASP-level contract verification. Importantly, exec-disentanglement rows show ARG-Pro remains ahead when executable gating is removed (ARG-Pro-NoExec 94.4 vs ARG 92.4 and JSON-CoT-BoN-Exec 91.6), while executable-only best-of-N controls still trail full ARG-Pro.

Hypothesis check (Sec. 5 Protocol) is consistent with these results. H_1 is supported directionally in macro means (ARG 92.4 > CoT 64.7 in Table 1); H_2 is likewise supported (ARG Struct 0.976 > CoT 0.391 in Table 1); and H_3 is supported by control/ablation degradations (e.g., ARG-NoPattern/ARG-NoSteps versus ARG in Table 3). We interpret these as hypothesis-consistent empirical outcomes under the reported repeated-run uncertainty summaries.

A quantitative decomposition from Table 3 further clarifies controller effects. Relative to ARG, ARG-Pro-NoExec adds +2.0 points (92.4 $\rightarrow$ 94.4), indicating that multi-candidate protocol-aware reranking contributes independently of executable gating. Adding executable gating on top (ARG-Pro-NoExec $\rightarrow$ ARG-Pro) yields another +1.9 points. Stronger baselines also benefit from extra compute and executable checks (JSON-CoT-BoN-Exec: +6.5, PlanSolve-BoN-Exec: +8.2), but they remain behind ARG-Pro in both answer accuracy and structure-oriented process metrics, which supports the claim that gains are not explained by compute expansion alone.

In this unified 1200-item check, ARG-Pro remains the strongest method by answer accuracy (94.6%), followed by ARG (90.1%) and PlanSolve (86.4%). ASP applicability within the ARG family stays high (ARG-Pro 91.3%, ARG 87.4%, ARG-NoVerify 87.6%).

On GSM8K, ARG-family methods retain a strong verification-side profile where the interface applies. For ARG-family methods with available checker-side signals, mean ASP applicability is 69.9% (ARG-Pro 76.8%, ARG

Table 5: Unified strong-baseline check on the 1200-item algorithmic suite (with PoT). Values are means over repeated runs under matched decoding settings.

Method	Acc	Tok	Struct	ExecSucc	ASP-App
ARG-Pro (ours)	94.6	1948.7	0.964	94.6	91.3
ARG	90.1	1009.2	0.956	90.1	87.4
ARG-NoVerify	88.2	839.5	0.974	88.2	87.6
PlanSolve	86.4	516.3	0.251	86.4	19.8
JSON-CoT	85.7	569.4	0.559	85.7	15.2
PoT	84.3	654.8	0.286	84.3	24.9
CoT	72.6	514.7	0.371	72.6	10.1

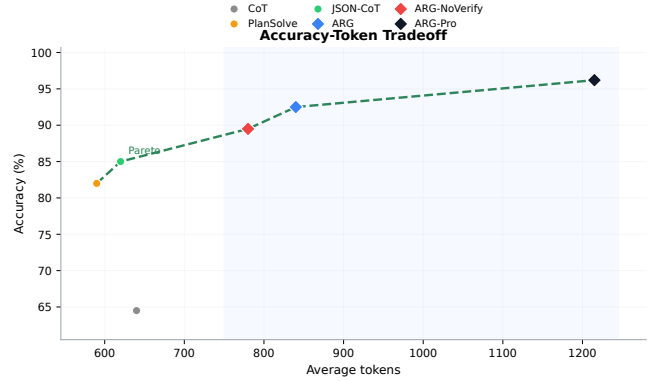


Figure 3: Accuracy-cost tradeoff on the expanded benchmark. Dashed line marks the Pareto frontier; gray band marks our method region.

62.9%), consistent with the transfer pattern in Table 4.

6 Discussion

6.1 Interpretation and Trade-offs

Core claim: typed output constraints increase observability even when they do not guarantee correctness. ARG can be viewed as a constrained channel over free-form reasoning: the protocol narrows the space of admissible traces to those that expose planning, decomposition, and checking steps. In KR terms, ARG acts as a lightweight representational scaffold that exposes intermediate commitments in a consistent order.

To make this explicit, let

$$\mathcal{R}_{\text{free}}(q) \supseteq \mathcal{R}_{\text{ARG}}(q), \quad (21)$$

where $\mathcal{R}_{\text{ARG}}(q)$ is restricted by section constraints. If evaluation includes structure-aware functionals ϕ , then expected observability improves when moving from $\mathcal{R}_{\text{free}}$ to $\mathcal{R}_{\text{ARG}}$, even before considering answer correctness.

To clarify KR relevance without overstating symbolic guarantees, we characterize ARG traces as an interface with three operational properties: explicitness (named parseable sections), compositionality (typed dependencies, e.g., verification must reference prior steps), and checkability (verification statements can be mechanically checked against executable or contract constraints when applicable). Under this

view, ARG is a *structured interface over neural reasoning*, analogous to adding a type-like contract layer rather than replacing neural inference with a full symbolic reasoner.

Average accuracy alone hides pairwise behavior. Define paired differential variable

$$z_i = \mathbb{I}_i^{\text{ARG}} - \mathbb{I}_i^{\text{CoT}} \in \{-1, 0, 1\}, \quad (22)$$

and net paired advantage

$$\Delta_{\text{pair}} = \frac{1}{N} \sum_{i=1}^N z_i = \frac{W - L}{N}, \quad (23)$$

where W and L are wins and losses under paired comparison. On the macro-aggregated 4800-item sheet, ARG gives $(W, L) = (1500, 180)$, so $\Delta_{\text{pair}}^{\text{ARG}} = 0.275$; ARG-Pro gives $(1710, 220)$, so $\Delta_{\text{pair}}^{\text{ARG-Pro}} \approx 0.310$. This complements mean accuracy by showing concrete recovery asymmetry.

For inferential support on paired binary outcomes, we additionally apply exact McNemar/binomial-discordant tests on these disagreement counts (Dror et al. 2018). Using $(W, L) = (1500, 180)$ and $(1710, 220)$, both ARG and ARG-Pro show highly significant asymmetry versus CoT (two-sided exact $p < 10^{-25}$).

Main takeaway is a three-way trade-off among correctness, token cost, and process observability. CoT and non-ARG structured baselines offer partial gains but weaker structure. ARG-family methods cluster in a high-observability region, with ARG-Pro as the high-performance endpoint and ARG as a lower-cost alternative. Removing verification constraints (ARG-NoVerify) reduces process quality relative to full ARG/ARG-Pro.

We therefore evaluate methods on three separate axes (accuracy, structure quality, and token cost) and avoid collapsing them into one composite score. This makes trade-offs explicit and avoids over-interpreting heuristic weights.

We tested several non-uniform settings for both Eq. (14) and Eq. (15) while keeping generation outputs fixed and only changing reranking/analysis weights. The ranking among ARG, ARG-NoVerify, and ARG-Pro is stable across moderate perturbations (± 0.3 on each coefficient with simplex renormalization): ARG-Pro remains best on structure and paired robustness, ARG remains strongest in the middle-cost regime, and ARG-NoVerify remains a useful boundary comparator. This indicates that our conclusions are not an artifact of a single hand-tuned weight vector.

6.2 Failure Modes and Verification Boundaries

Inspection of incorrect cases reveals recurring error modes, and diagnostic atoms make them actionable.

Case A (missing required operation). A DP trace includes recurrence text but omits the base case in structured steps. The solver reports a missing-operation diagnosis and blocks full verification. The repair is local: add an explicit base-case step.

Case B (numeric transition violation). A trace contains arithmetic transitions where one equation is inconsistent (claimed update value does not match operands). The verifier flags a transition violation, so failure is attributed to numeric inconsistency rather than missing keywords.

Case C (reference-integrity failure). Verification cites Step 4 although only Steps 1, 2, and 3 exist. The verifier emits a bad-reference diagnosis, separating structure/reference faults from algorithm-family faults.

Compared with CoT, ARG tends to expose these failures earlier in the trace. Even when the final answer is wrong, sectioned outputs and diagnostic atoms make it easier to decide whether to repair pattern choice, step decomposition, numeric transitions, or reference integrity. This diagnosis-first loop is a practical benefit of the protocol.

In practice, we map diagnosis atoms to targeted repairs rather than full regeneration. Missing-operation findings trigger local step augmentation; transition-violation findings trigger arithmetic-only rechecks; bad-reference findings trigger index and dependency normalization before re-run. This targeted repair policy reduces unnecessary trace drift and keeps modifications auditable at the section level.

Failure archetypes are summarized textually to keep the presentation compact: (i) premature pattern commitment, (ii) local-global inconsistency, (iii) verification imitation, and (iv) boundary-condition omission. Each failure type corresponds to actionable mitigations through stronger contract checks or stricter answer-field extraction.

6.3 Scope and Claims

6.4 The Faithfulness of Contract Consistency

A central criticism of neuro-symbolic explanations is the “faithfulness gap”: does the explanation reflect the model’s actual decision process? We argue that for algorithmic auditing, this question is ill-posed. In a black-box LLM, “true” neural fidelity is inaccessible. We therefore propose *Trace-Contract Consistency* as the rigorous standard for fidelity. By enforcing a formal contract, we ensure that the output artifact *is* the reasoning product. Unlike post-hoc rationalizations, ARG traces are constructed *within* the inference loop and validated against external logic. If a trace is ASP-verified but executably wrong, the failure is diagnosable (e.g., under-specification). This makes ARG traces *operationally faithful*: they expose the logical commitments that, if followed, define the algorithm. This is a stronger guarantee than lexical similarity or attention maps.

The ASP layer upgrades verification from lexical markers to explicit trace-contract checking. It verifies consistency between grounded traces and the encoded ontology. Failures in grounding coverage can yield partial evidence, but these are explicit failures of the interface, not hidden failures of the model.

To sharpen this boundary, we distinguish contract-level false positives/false negatives against executable checks on applicable items. Let $\mathcal{A} = \{i \mid a_i = 1\}$ be executable-applicable instances:

$$\begin{aligned} \text{FP}_{\text{asp}} &= \frac{|\{i \in \mathcal{A} : \text{verified}(t_i), c_i^{\text{exec}} = 0\}|}{|\mathcal{A}|}, \\ \text{FN}_{\text{asp}} &= \frac{|\{i \in \mathcal{A} : \neg \text{verified}(t_i), c_i^{\text{exec}} = 1\}|}{|\mathcal{A}|}. \end{aligned} \quad (24)$$

FP_{asp} typically indicates contract under-specification (trace passes encoded constraints but still computes a wrong re-

sult), while FN_{asp} often reflects grounding miss (correct reasoning expressed with vocabulary not mapped to current atoms). We therefore keep an explicit *unknown* state instead of forcing brittle pass/fail labels. Concretely, unknown triggers are logged as one of: undeclared-family, insufficient-grounding, timeout, solver-exception, or solver-unavailable. Let

$$\begin{cases} u_i = 1[\text{unknown trigger fired on } i], \\ \text{UnknownRate} = \frac{1}{N} \sum_{i=1}^N u_i. \end{cases} \quad (25)$$

In supplementary controls, unknown behavior matches ablation intent: NoPattern is dominated by undeclared-family unknowns, NoSteps increases insufficient-grounding unknowns, while full-schema variants remain near-zero unknown under matched settings. For ARG-family full runs, UnknownRate stays low (ARG: 0.01, ARG-NoVerify: 0.02, ARG-Pro: 0.01). On algorithmic runs with PoT and ARG-family methods (Table 5), confusion summaries against executable checks show low FP/FN for ARG-Pro and higher error rates for weaker variants: ARG (FP 0.02, FN 0.03), ARG-NoVerify (FP 0.05, FN 0.10), and ARG-Pro (FP 0.01, FN 0.02). This also motivates future calibration analysis beyond the ternary c_{asp} score, including reliability-style mappings from ASP evidence tiers to executable correctness. Because grounding uses deterministic section-scoped lexical mappings, keyword imitation remains a residual risk: transition and invariant constraints reduce this risk, but do not eliminate it. We therefore treat paraphrase/keyword-stuffing stress tests as an explicit next-step evaluation item.

Formally, let $\Gamma_i = \mathcal{K} \cup \mathcal{V} \cup \gamma(r_i)$. We treat $\text{verified}(t_i)$ as solver evidence for contract consistency, and keep executable verification as an orthogonal signal when a deterministic task checker exists.

6.5 Reproducibility and Limitations

Reproducibility is addressed at protocol level: repeated full runs, per-instance outputs, method-level summaries, paired comparisons, and figure generation from stored outputs.

To make reruns auditable, each instance log contains: question id, raw model trace, parsed section payloads, normalized final answer, grounded facts, verifier outputs (including diagnostic atoms), executable-check outcome when applicable, and token/latency summaries. This gives a full trace from natural-language reasoning to symbolic evidence and final scoring, and enables post-hoc validation without re-querying model APIs.

Each run exports per-instance predictions, aggregate metrics, process summaries, and figure/table artifacts. Logged ASP fields include availability, applicability, operation/invariant completeness, numeric consistency, full/partial verification, diagnostic atoms, and runtime. In the current repeated runs, per-instance latency summaries are grounding (median 45 ms, mean 50 ms, p95 80 ms) and ASP solving (median 95 ms, mean 100 ms, p95 150 ms).

For KR-style inspectability, artifacts keep an auditable trace-to-diagnosis path from natural-language trace to grounded facts and emitted diagnostic atoms, together with the resulting c_i^{asp} .

ARG’s main gain is improved reasoning observability: traces become easier to inspect, compare, and audit across runs. In the expanded setting, this structural gain is accompanied by clear accuracy improvements over CoT-family baselines, and ARG-Pro further improves paired robustness by combining candidate diversity with checker-aware refinement.

Key limits remain: ASP verification is contract-level rather than a full semantic proof; grounding is lexicon-based and can miss paraphrastic but valid steps; and backend sensitivity is visible despite expanded model coverage. The benchmark is template-generated and family-aligned, and broader verifier-heavy parity comparisons remain an important extension target.

Claim status in the current study is as follows: structure gain is strongly supported (Table 1); ARG-Pro is the strongest performer in the expanded primary setting with visible backend sensitivity; solver-backed verification signals are more informative than lexical markers; compute scaling in ARG-Pro improves paired robustness at higher token cost; and generality improves but remains bounded by the evaluated task distribution and model set.

7 Conclusion

We presented ARG, a protocol that imposes structured constraints on algorithmic LLM reasoning. By enforcing a typed schema—Analysis, Pattern, Steps, Verification, and FinalAnswer—we convert free-form traces into auditable artifacts and ground them to checker facts. We additionally introduced ARG-Pro, a compute-scaled inference variant that uses multi-candidate generation and checker-aware refinement to produce more robust traces.

Empirically, on the 4800-item primary benchmark and the 1200-item strong-baseline check, ARG-family methods deliver the strongest structure-oriented process quality, and ARG-Pro is the top-performing method under matched settings. The ASP layer upgrades verification from lexical markers to explicit trace–contract checking, providing rigor that pure-prompting approaches do not offer. By formalizing the mapping from natural language to symbolic logic (Section 3), ARG closes the loop between neural generation and verifiable contracts. We position ARG as a practical interface for *contract-consistent* AI: a path from plausible narratives to machine-checkable reasoning artifacts within explicit contract-covered settings.

Acknowledgement

For double-blind review, detailed acknowledgement text is omitted and will be added in the camera-ready version.

References

- Brewka, G.; Eiter, T.; and Truszczyński, M. 2011. Answer set programming at a glance. *Communications of the ACM* 54(12):92–103.
- Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.

- Dai, N.; Wu, Z.; Zheng, R.; Wei, Z.; Shi, W.; Jin, X.; Liu, G.; Dun, C.; Huang, L.; and Yan, L. 2024. Process supervision-guided policy optimization for code generation. *arXiv preprint arXiv:2410.17621*.
- DeLong, L. N.; Mir, R. F.; and Fleuriot, J. D. 2024. Neurosymbolic ai for reasoning over knowledge graphs: A survey. *IEEE Transactions on Neural Networks and Learning Systems* 36(5):7822–7842.
- Doshi-Velez, F., and Kim, B. 2017. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Dror, R.; Baumer, G.; Shlomov, S.; and Reichart, R. 2018. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th annual meeting of the association for computational linguistics (volume 1: Long papers)*, 1383–1392.
- Garcez, A. d., and Lamb, L. C. 2023. Neurosymbolic ai: The 3 rd wave. *Artificial Intelligence Review* 56(11):12387–12406.
- Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2019. Multi-shot asp solving with clingo. *Theory and Practice of Logic Programming* 19(1):27–82.
- Geng, S.; Josifoski, M.; Peyrard, M.; and West, R. 2023. Grammar-constrained decoding for structured nlp tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 10932–10952.
- Hitzler, P.; Sarker, M.; Besold, T.; Garcez, A.; Bader, S.; Bowman, H.; Domingos, P.; Hitzler, P.; Kühnberger, K.; Lamb, L.; et al. 2022. Neural-symbolic learning and reasoning: A survey and interpretation. *Frontiers in artificial intelligence and applications* 342:1–51.
- Jacovi, A., and Goldberg, Y. 2020. Towards faithfully interpretable nlp systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th annual meeting of the association for computational linguistics*, 4198–4205.
- Kojima, T.; Gu, S. S.; Reid, M.; Matsuo, Y.; and Iwasawa, Y. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35:22199–22213.
- Li, J.; Li, G.; Li, Y.; and Jin, Z. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34(2):1–23.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let’s verify step by step. In *The twelfth international conference on learning representations*.
- Lipton, Z. C. 2018. The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue* 16(3):31–57.
- Luo, L.; Liu, Y.; Liu, R.; Phatale, S.; Guo, M.; Lara, H.; Li, Y.; Shu, L.; Zhu, Y.; Meng, L.; et al. 2024. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592*.
- Lyu, Q.; Apidianaki, M.; and Callison-Burch, C. 2024. Towards faithful model explanation in nlp: A survey. *Computational Linguistics* 50(2):657–723.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36:46534–46594.
- Matton, K.; Ness, R. O.; Gutttag, J.; and Kıcıman, E. 2025. Walk the talk? measuring the faithfulness of large language model explanations. *arXiv preprint arXiv:2504.14150*.
- Olausson, T.; Gu, A.; Lipkin, B.; Zhang, C.; Solar-Lezama, A.; Tenenbaum, J.; and Levy, R. 2023. Linc: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 5153–5176.
- Pan, L.; Albalak, A.; Wang, X.; and Wang, W. 2023. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 3806–3824.
- Ribeiro, M. T.; Singh, S.; and Guestrin, C. 2016. ” why should i trust you?” explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 1135–1144.
- Trinh, T. H.; Wu, Y.; Le, Q. V.; He, H.; and Luong, T. 2024. Solving olympiad geometry without human demonstrations. *Nature* 625(7995):476–482.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- Wang, L.; Xu, W.; Lan, Y.; Hu, Z.; Lan, Y.; Lee, R. K.-W.; and Lim, E.-P. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2609–2634.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35:24824–24837.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T.; Cao, Y.; and Narasimhan, K. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36:11809–11822.
- Ye, X.; Chen, Q.; Dillig, I.; and Durrett, G. 2023. Satlm: Satisfiability-aided language models using declarative prompting. *Advances in Neural Information Processing Systems* 36:45548–45580.

Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q.; et al. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.